Харківський національний університет радіоелектроніки
Факультет комп’ютерних наук
Катедра програмної інженерії

ЗВІТ
до практичного завдання № 1
з дисципліни «Аналіз та рефакторинґ коду»
на тему: «Основні рекомендації з написання коду мовою Java»

Виконав
ст. гр. ПЗПІ-24-6
Кулініч Олександр Олександрович

Перевірив
ст. викладач катедри ПІ
Сокорчук Ігор Петрович

Харків 2026

1 ІСТОРІЯ ЗМІН

№	Дата	Версія звіту	Опис змін та виправлень
	21.10.2026	0.1	Створено розділ «Завдання»
2	22.10.2026	0.1	Створено розділ «Опис виконаної роботи»
3	09.11.2026	1.0	Оформлено додатки

2 ЗАВДАННЯ

Метою першої практичної роботи є дослідження основних рекомендацій щодо написання якісного та чистого коду в Java. На практиці показати рекомендації, та як вони впливають на розуміння написаного, та його легкість у маштабуванні.

Основні завдання:

1. ознайомитись зі стандартами кодування мови програмування Java;
2. дослідити основні рекомендації написання чистого коду на Java: стильові рекомендації, структури коду, принципи його рефакторингу, обробку помилок, тестування та документування коду;
3. підготувати приклади коду, що демонструють застосування рекомендацій;
4. підготувати презентацію з 12 слайдів;
5. записати відео доповіді тривалістю 7-9 хвилин;
6. оформити звіт українською мовою, включити опис виконання роботи, висновки, список джерел та додатки А, Б, В.

3 ОПИС ВИКОНАНОЇ РОБОТИ

Для доповіді було обрано мову програмування Java. Це строго типізована, об’єктно-орієнтована мова програмування з автоматичним керуванням пам’яттю(GC), що виконується у віртуальній машині JVM.

Java широко використовується для розробки серверних рішень, корпоративних систем та Android-застосунків. Через велику кодову базу та тривалий життєвий цикл Java-проєктів єдині правила оформлення коду мають особливе значення: код читають і змінюють набагато частіше, ніж пишуть.

3.1 Використовуйте осмислені назви змінних, методів і класів

Правильне іменування допомагає іншим розробникам легко зрозуміти призначення того чи іншого шматка коду. Уникайте абревіатур і коротких імен без значення. У Java діють усталені угоди: класи та інтерфейси — UpperCamelCase, методи та змінні — lowerCamelCase, константи — UPPER_SNAKE_CASE, пакети — малими літерами без підкреслень. Назви методів мають бути дієсловами, назви класів — іменниками.

Приклад:

```
// Поганий приклад
static int f(int a) {
    return a * 2;
}

// Гарний приклад
static int multiplyByTwo(int number) {
    return number * 2;
}

// Поганий приклад
class order_service {
    int Max_cnt = 10;
    double calc(double a, double b) { return a * b; }
}

// Гарний приклад
class OrderService {
    private static final int MAX_ITEMS_COUNT = 10;

    double calculateTotalPrice(double unitPrice, double quantity) {
        return unitPrice * quantity;
```

```
23      }
24  }
```

Компілятор Java не вимагає дотримання угод, але багато бібліотек, фреймворків та IDE покладаються на них. Наприклад JavaBeans розпізнає властивості за іменами getX/setX. Або у Inteliji Idea при натисканні Alt + Insert відкриєтиься генератор, який може створювати методи по типу getX/setX, та не давати створити дуплікати, якщо такі методи вже існують. Недотримання угод змушує читача витрачати час на розшифрування змісту, збільшує ймовірність помилок під час змін і ускладнює освоєння нових членів команди у кодовій базі. Змістовні імена роблять коментарі здебільшого непотрібними.

3.2 Дотримуйтесь єдиного форматування коду

Цей пункт про дотримання єдиних відступів у коді, обмеження довжини рядка до 80-100 символів, та не писати складні логічні конструкції у один рядок, одна інструкція в одному рядку та фігурні дужки навіть для однорядкових блоків if, for, while.

Приклад:

```
 1  // Поганий приклад
 2  if (user != null) return user.getName(); else return "unknown";
 3  for (int i = 0; i < n; i++) sum += values[i];
 4
 5  // Гарний приклад
 6  if (user != null) {
 7      return user.getName();
 8  } else {
 9      return "unknown";
10  }
11
12  for (int i = 0; i < n; i++) {
13      sum += values[i];
14  }
```

Відсутність фігурних дужок, це доволі часте джерело помилок, наприклад під час додавання другого рядка до тіла if розробник може помилково вважати, що він також належить умові. Також одна інструкція в одному рядку сприяє код ревью через систему контролю версій. Форматування варто автоматизувати інструментами (google-java-format, форматер IDE).

3.3 Дотримуйтесь структури файлу та пакетів

Один публічний клас на один файл, назва файлу збігається з назвою класу. Порядок оголошень у файлі: спочатку package, далі імпорти, далі клас. Не використовувати імпорти з зірочкою, прибирати невикористані імпорти. Усередині класу: константи, поля, конструктори, публічні методи, приватні методи. Пакети організовуйте за призначенням (com.example.shop.order, com.example.shop.payment), а не за технічним типом усього підряд.

Приклад:

```
// Поганий приклад
import java.util.*;
import java.io.*;

public class Shop {
    void process() { List<String> items = new ArrayList<>(); }
    private int count;
    public static final int LIMIT = 5;
}

// Гарний приклад
package com.example.shop.order;

import java.util.ArrayList;
import java.util.List;

public class OrderProcessor {
    public static final int ITEMS_LIMIT = 5;

    private int processedCount;

    public void process() {
        List<String> items = new ArrayList<>();
        // ...
    }
}
```

Явні імпорти показують залежності класу та запобігають конфліктам імен (наприклад, java.util.Date і java.sql.Date). Передбачуваний порядок елементів дозволяє швидко знаходити потрібне в будь-якому класі проєкту.

3.4 Пишіть короткі методи з однією відповідальністю

Метод має виконувати одну задачу на одному рівні абстракції. Якщо метод перевищує екран, містить глибоку вкладеність або коментарі-розділювачі «крок 1, крок 2», його слід розбити, винісши код по методам, за допомогою «Виділення методу» [5]. Обмежуйте кількість параметрів (бажано не більше 3–4 на метод), використовуйте «раннє повернення» замість глибокої вкладеності.

Приклад:

```
// Поганий приклад
void processOrder(Order order) {
    if (order != null) {
        if (!order.getItems().isEmpty()) {
            double total = 0;
            for (Item item : order.getItems()) {
                total += item.getPrice() * item.getQuantity();
            }
            if (total > 1000) {
                total = total * 0.9;
            }
            order.setTotal(total);
        }
    }
}

// Гарний приклад
void processOrder(Order order) {
    if (order == null || order.getItems().isEmpty()) {
        return;
    }
    double total = calculateTotal(order.getItems());
    order.setTotal(applyDiscount(total));
}

private double calculateTotal(List<Item> items) {
    double total = 0;
    for (Item item : items) {
        total += item.getPrice() * item.getQuantity();
    }
    return total;
}

private double applyDiscount(double total) {
    return total > DISCOUNT_THRESHOLD ? total * DISCOUNT_RATE : total;
}
```

Малі методи легше читати, повторно використовувати й покривати модульними тестами: у прикладі calculateTotal та applyDiscount можуть тестуються окремо. Довгі методи з багатьма відповідальностями — основне джерело дефектів при змінах, адже зміна однієї частини логіки може непомітно

зачепити іншу. Принцип єдиної відповідальності знижує вартість подальшої підтримки.

3.5 Замінюйте «магічні числа» іменованими константами

Літерали, значення яких не очевидне з контексту, слід виносити в константи private static final або в enum. Рядкові та числові «коди статусів» краще замінювати переліченнями.

```
// Поганий приклад
if (order.getStatus() == 2 && total > 1000) {
    total = total * 0.9;
}

// Гарний приклад
private static final double DISCOUNT_THRESHOLD = 1000.0;
private static final double DISCOUNT_RATE = 0.9;

enum OrderStatus { NEW, PAID, SHIPPED, CANCELLED }

if (order.getStatus() == OrderStatus.PAID && total > DISCOUNT_THRESHOLD) {
    total = total * DISCOUNT_RATE;
}
```

Читач не може здогадатися, що означає 2 чи 0.9, а зміна значення вимагає пошуку всіх входжень по коду — легко пропустити одне з них. enum у Java є типобезпечним: компілятор не дозволить передати довільне число замість статусу, а оператор switch над enum може перевірятися на повноту.

3.6 Правильно обробляйте помилки

Перехоплюйте конкретні типи винятків, а не Exception чи Throwable; ніколи не залишайте порожній блок catch; не використовуйте винятки для керування потоком виконання; для ресурсів застосовуйте try-with-resources; перевіряйте аргументи на початку методу (Objects.requireNonNull, IllegalArgumentException);

Приклад:

```
// Поганий приклад
BufferedReader reader = null;
try {
reader = new BufferedReader(new FileReader(path));
        return reader.readLine();
} catch (Exception e) {
        // ignore
        } finally {
        try { reader.close(); } catch (Exception e) { }
        }
        return null;

// Гарний приклад
        try (BufferedReader reader = new BufferedReader(new FileReader(path))) {
        return reader.readLine();
} catch (IOException e) {
        throw new ConfigLoadException("Не вдалося прочитати файл: " + path, e);
}
```

Проковтнуті винятки приховують реальну причину збою: програма продовжує працювати з некоректним станом, а помилка проявляється пізніше й у іншому місці, що значно ускладнює налагодження. try-with-resources гарантує закриття ресурсу навіть у разі виняткової ситуації та усуває громіздкі і схильні до помилок вкладені блоки finally. Збереження cause дає повний стек викликів у логах.

3.7 Віддавайте перевагу незмінності та сучасним засобам мови

Робіть поля final, де це можливо. Не надавайте сеттери без потреби; для простих об’єктів-обгорток для даних використовуйте record. Замість null для «відсутнього значення» застосовуйте Optional як тип результату методу; використовуйте var лише там, де тип очевидний із правої частини.

Приклад:

```
// Поганий приклад
public class Point {
    private int x;
    private int y;

    public int getX() { return x; }
    public void setX(int x) { this.x = x; }
    public int getY() { return y; }
    public void setY(int y) { this.y = y; }
```

```
10      // + equals(), hashCode(), toString() вручну
11  }
12
13  // Гарний приклад
14  public record Point(int x, int y) { }
15
16  // Повернення Optional замість null
17  public Optional<User> findUserByEmail(String email) {
18      return Optional.ofNullable(users.get(email));
19  }
```

Незмінні об’єкти потокобезпечні за своєю природою, їх безпечно використовувати як ключі у HashMap і передавати між компонентами без захисного копіювання [3]. record автоматично генерує конструктор, equals, hashCode і toString, зменшуючи обсяг шаблонного коду та ймовірність розбіжності цих методів. Optional явно сигналізує у сигнатурі методу, що результату може не бути, і знижує ризик NullPointerException.

3.8 Програмуйте відносно інтерфейсів і використовуйте дженеріки

Типи змінних, полів і параметрів оголошуйте через інтерфейси (List, Map, Set), а не через конкретні реалізації. Не використовуйте «сирі» типи (raw types) — завжди вказуйте параметри типу. Надавайте перевагу композиції над успадкуванням.

Приклад:

```
1  // Поганий приклад
2  ArrayList items = new ArrayList();
3  items.add("apple");
4  String first = (String) items.get(0);
5
6  // Гарний приклад
7  List<String> items = new ArrayList<>();
8  items.add("apple");
9  String first = items.get(0);
```

Узагальнення переносять перевірку типів із часу виконання на час компіляції: помилка ClassCastException стає помилкою компіляції. Оголошення через інтерфейс дозволяє змінити реалізацію (наприклад, ArrayList на

LinkedList чи CopyOnWriteArrayList) без змін в усьому коді, що знижує зв’язність, та спрощує модифікацію коду.

3.9 Враховуйте продуктивність там, де це справді важливо

Не застосовуйте передчасну оптимізацію, але уникайте відомих неефективних конструкцій: конкатенація рядків у циклі (краще використати StringBuilder), зайве автоупаковування (Long замість long), повторне обчислення незмінних значень усередині циклу, вибір відповідної колекції (HashMap для пошуку за ключем замість перебору List).

Приклад:

```
// Поганий приклад
String result = "";
for (String word : words) {
    result += word + ", ";
}

Long sum = 0L;
for (int i = 0; i < 1_000_000; i++) {
    sum += i;
}

// Гарний приклад
StringBuilder builder = new StringBuilder();
for (String word : words) {
    builder.append(word).append(", ");
}
String result = builder.toString();

long sum = 0L;
for (int i = 0; i < 1_000_000; i++) {
    sum += i;
}
```

Рядки в Java незмінні, тому кожна конкатенація в циклі створює новий об’єкт і копіює вміст, що дає квадратичну складність за довжиною результату. Автоупаковування в циклі створює мільйони тимчасових об’єктів і навантажує GC. Водночас оптимізувати слід на основі вимірювань профілювальнику, а не інтуїції, щоб не погіршувати читабельність без користі [3].

3.10 Документуйте код за допомогою Javadoc

Усі публічні класи, інтерфейси та методи слід документувати коментарями Javadoc: короткий опис призначення, @param, @return, @throws. Коментар має пояснювати «чому» та контракт методу, а не повторювати те, що й так зрозуміло з коду.

Приклад:

```
// Поганий приклад
// множимо
public double calc(double a, double b) {
    return a * b; // повертаємо результат
}

// Гарний приклад
/**
 * Обчислює вартість позиції замовлення.
 *
 * @param unitPrice ціна за одиницю товару, не може бути від'ємною
 * @param quantity  кількість одиниць, додатне число
 * @return загальна вартість позиції
 * @throws IllegalArgumentException якщо ціна або кількість недопустимі
 */
public double calculateItemCost(double unitPrice, double quantity) {
    if (unitPrice < 0 || quantity <= 0) {
        throw new IllegalArgumentException("Некоректні ціна або кількість");
    }
    return unitPrice * quantity;
}
```

Javadoc автоматично перетворюється на HTML-документацію API та відображається у підказках IDE, тому є частиною контракту класу [7]. Коментарі, що дублюють код, швидко застарівають і вводять в оману. Задокументовані вимоги до аргументів і винятків знижують кількість помилок під час використання чужого коду.

3.11 Покривайте код модульними тестами

Для Java стандартом є Junit [6]. Тест має бути ізольованим, детермінованим, швидким і перевіряти одну поведінку. Назва тесту описує сценарій і очікуваний результат; структура — Arrange / Act / Assert.

Приклад:

```
 1  import static org.junit.jupiter.api.Assertions.assertEquals;
 2  import static org.junit.jupiter.api.Assertions.assertThrows;
 3
 4  import org.junit.jupiter.api.Test;
 5
 6  class OrderServiceTest {
 7
 8      private final OrderService service = new OrderService();
 9
10      @Test
11      void calculateItemCost_validInput_returnsProduct() {
12          double result = service.calculateItemCost(10.0, 3);
13
14          assertEquals(30.0, result, 0.0001);
15      }
16
17      @Test
18      void calculateItemCost_negativePrice_throwsException() {
19          assertThrows(IllegalArgumentException.class,
20                  () -> service.calculateItemCost(-1.0, 3));
21      }
22  }
```

Модульні тести є страховкою при рефакторингу, не завжди виходить с першого разу сзінити код, не змінивши його поведінку, і довести що код працює індентично попередньому можна лише автоматичною перевіркою [5].

3.12 Програмні інструменти для аналізу якості коду

Для автоматизації контролю дотримання рекомендацій у Java-проєктах використовуються такі інструменти:

- **PMD** — статичний аналіз: невикористані змінні, порожні блоки catch, надто складні методи(рис. 3.1.);

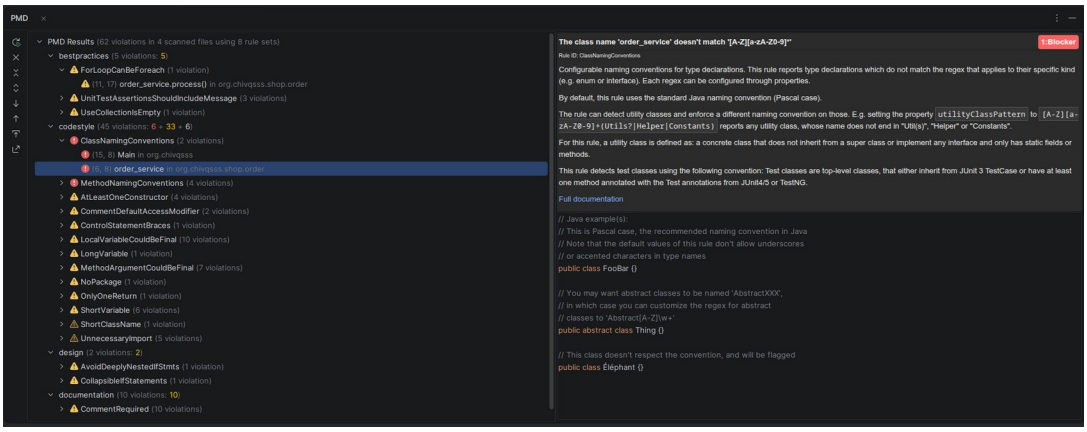


Рисунок 3.1 — PMD як розширення у Inteliji Idea.

- **Checkstyle** — перевірка відповідності стилю кодування (наприклад, Google Style або Sun Checks): іменування, відступи, імпорти, довжина рядків;
- **SpotBugs** — пошук потенційних дефектів у байт-коді (можливий NullPointerException, неправильне використання equals, тощо)
- **SonarQube / SonarLint** — комплексна оцінка якості: дублювання коду, цикломатична складність, «запахи коду», технічний борг;
- **google-java-format** та **форматер IntelliJ IDEA** — автоматичне форматування коду;
- **IntelliJ IDEA** — вбудовані інспекції та автоматичні рефакторинги (Extract Method, Rename, Inline, Change Signature, тощо);
- **JUnit** — тестування та вимірювання покриття коду тестами.

3.13 Практичний приклад рефакторингу

У додатках В.2 та В.3 наведено приклад коду до та після рефакторингу, що поєднує кілька розглянутих рекомендацій: осмислені імена, константи замість «магічних чисел», виділення методів, обробка вхідних даних, застосування record та enum. Поведінка програми при цьому не змінюється, що перевіряється модульними тестами (додаток В.4).

4 ВИСНОВКИ

Під час виконання роботи було розглянуто основні рекомендації написання коду мовою Java: стильові конвенції Google Java Style Guide та Oracle Code Conventions, структуру файлів і пакетів, іменування, форматування, обробку помилок, незмінність, документування за допомогою Javadoc, тестування та інструменти автоматичної перевірки.

Отримано навички оформлення Java-коду згідно зі стильовими конвенціями, використання констант та enum замість «магічних чисел», виділення коротких методів, застосування try-with-resources, record та Optional.

5 ВИКОРИСТАНІ ДЖЕРЕЛА

1. Google Java Style Guide. *Google Style Guides*. URL: https://github.com/google/styleguide/blob/gh-pages/javaguide.md (дата звернення: 21.10.2026).
2. Code Conventions for the Java Programming Language. *Oracle*. URL: https://www.oracle.com/java/technologies/javase/codeconventions-contents.html (дата звернення: 21.10.2026).
3. Bloch J. Effective Java. 3rd ed. Boston : Addison-Wesley, 2018.
4. Martin R. C. Clean Code: A Handbook of Agile Software Craftsmanship. Upper Saddle River : Prentice Hall, 2008.
5. Fowler M. Refactoring: Improving the Design of Existing Code. 2nd ed. Boston : Addison-Wesley, 2018.
6. JUnit 5 User Guide. URL: https://junit.org/junit5/docs/current/user-guide/ (дата звернення: 22.10.2026).
7. Javadoc Tool. *Oracle Java Documentation*. URL: https://docs.oracle.com/en/java/javase/21/javadoc/javadoc-tool.html (дата звернення: 22.10.2026).

ДОДАТОК А

Відеозапис

Відеозапис доповіді: https://youtu.be/V7AG5rm26mw

Хронологічний опис відеозапису:

00:00 — Вступ: обрана мова Java, її особливості

00:52 — іменування та форматування коду

02:39 — структура файлів, короткі методи, константи

04:15 — обробка помилок, незмінність, інтерфейси та generics

05:34 — продуктивність, Javadoc, тестування

07:26 — Інструменти аналізу якості коду

9:16 — Висновки

09:55 — Демонстрація рефакторингу прикладу до/після

ДОДАТОК Б

Слайди презентації



Рисунок Б.1 — Титульний слайд: тема та обрана мова програмування Java

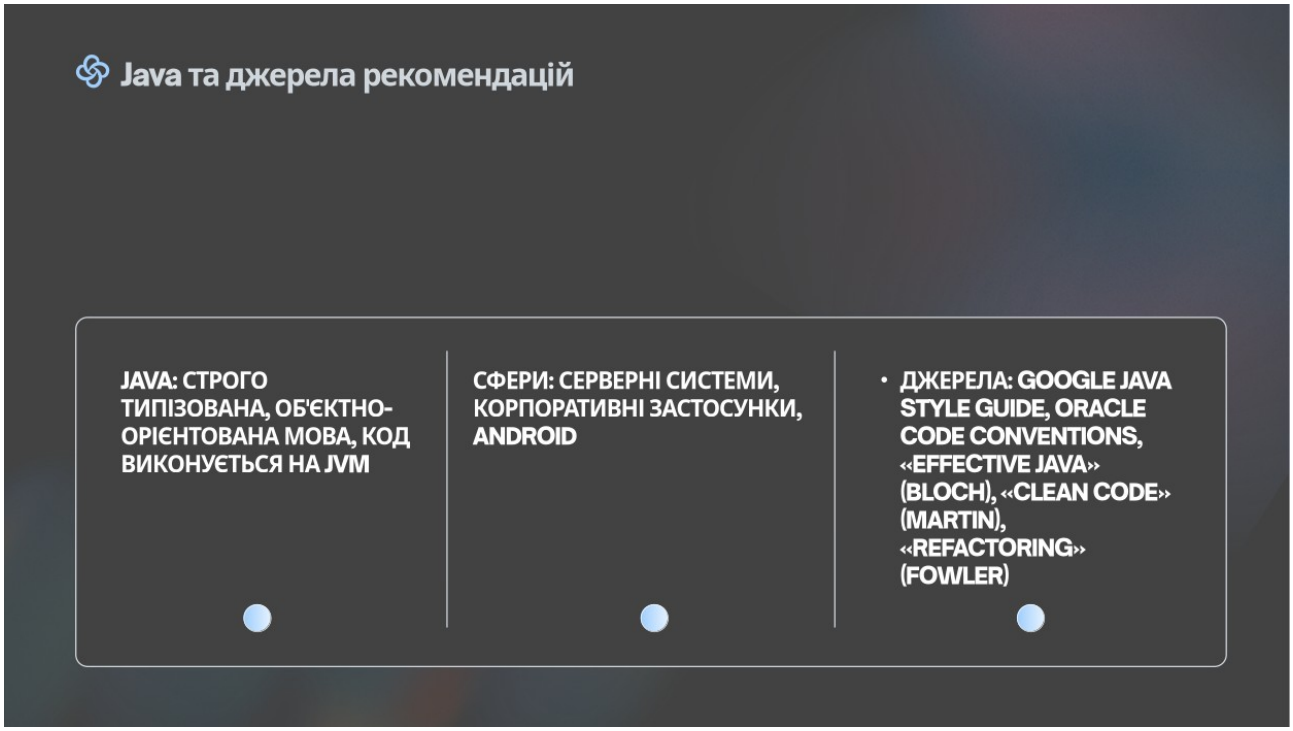


Рисунок Б.2 — Огляд особливостей мови Java та джерел рекомендацій

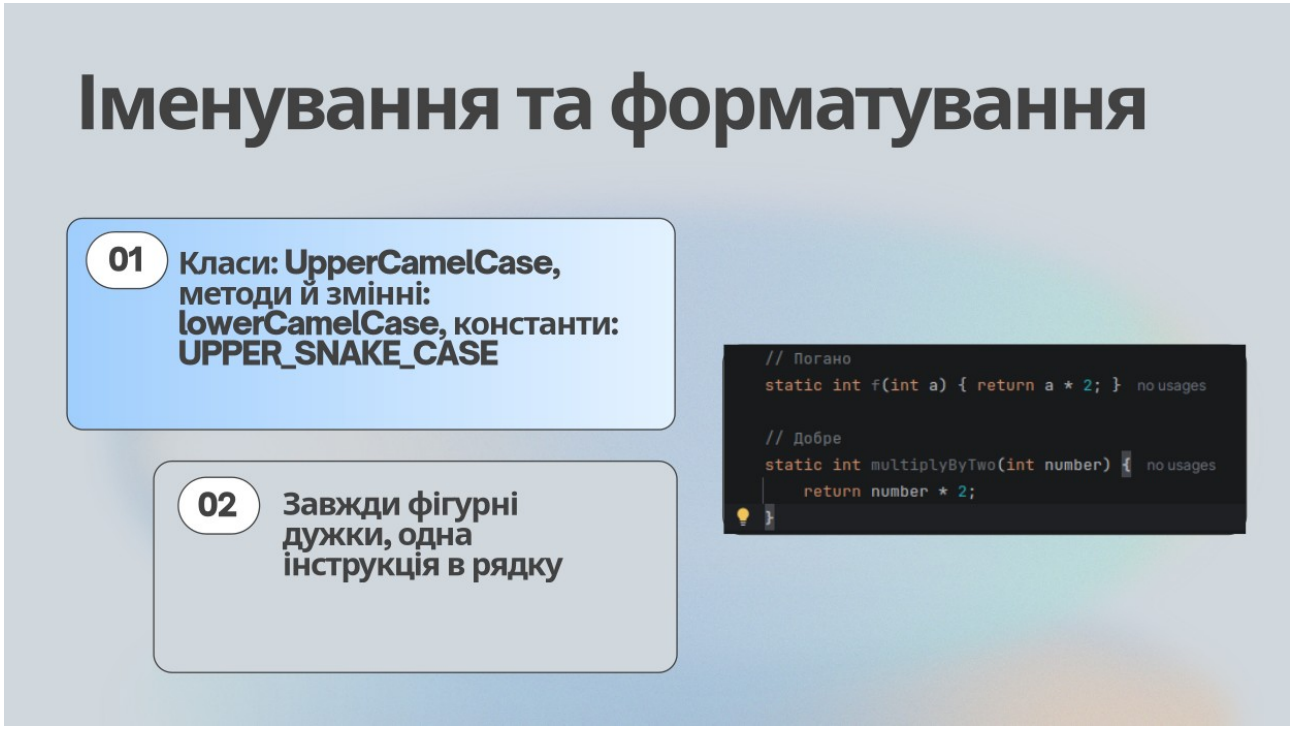


Рисунок Б.3 — Рекомендації з іменування та форматування коду

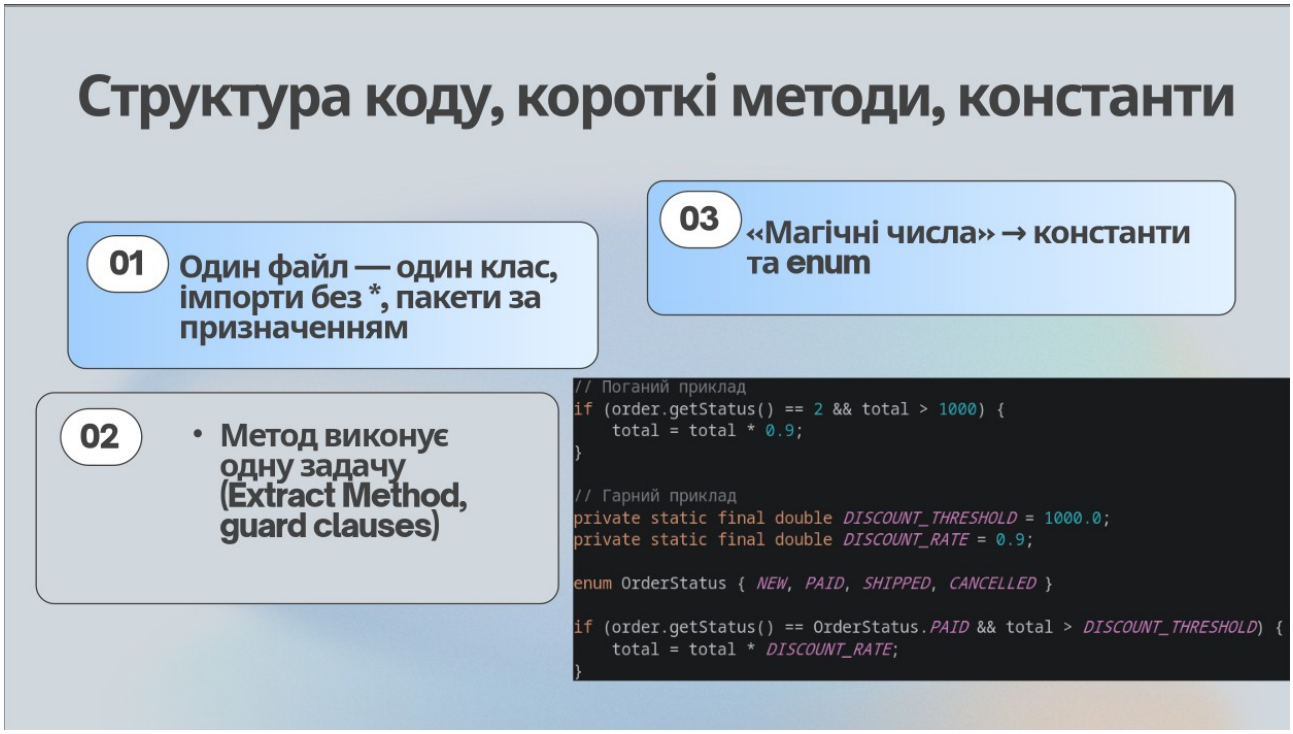


Рисунок Б.4 — Структура коду, короткі методи, константи

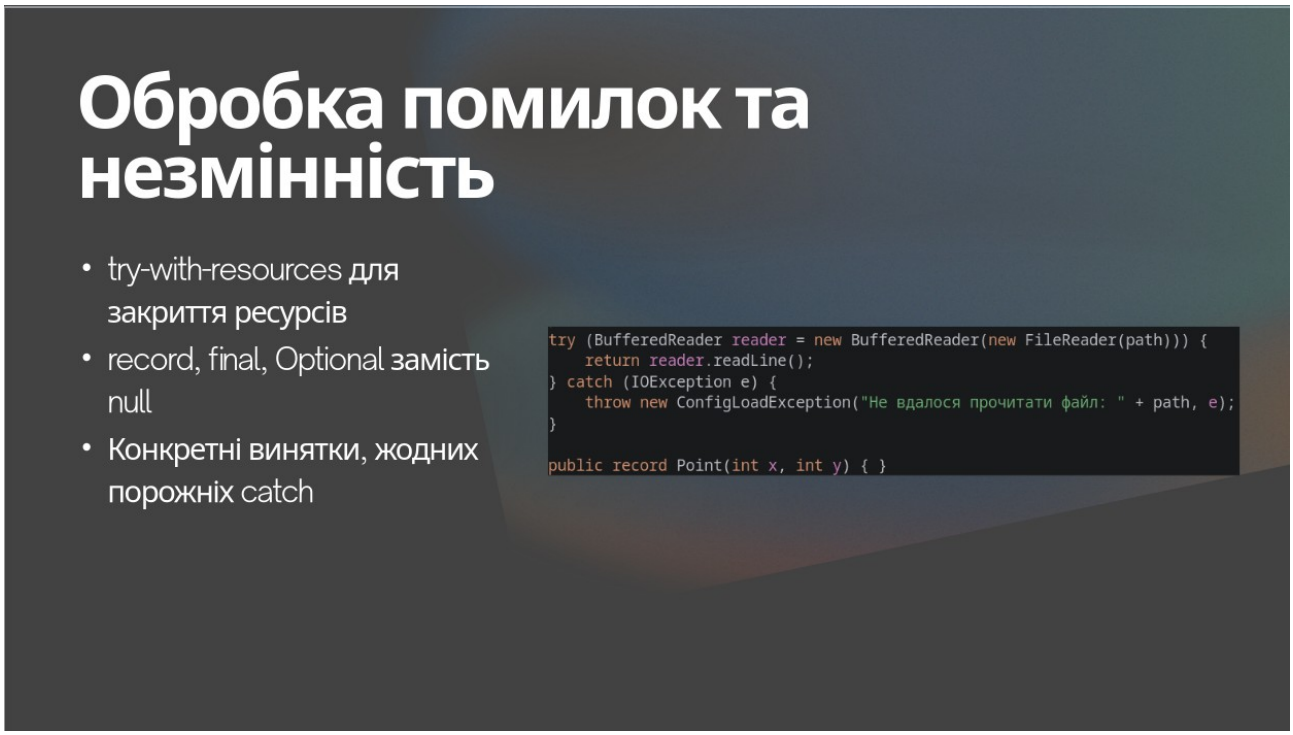


Рисунок Б.5 — Обробка помилок та незмінність

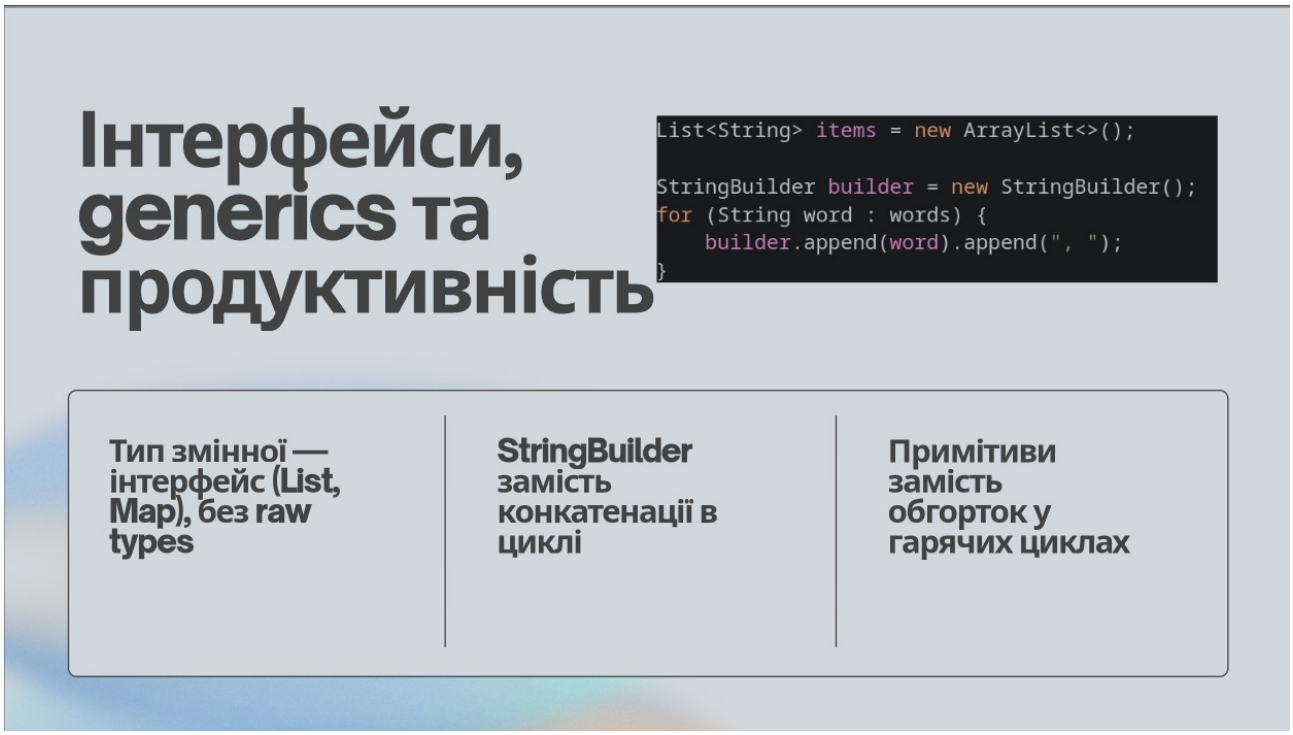


Рисунок Б.6 — Інтерфейси, generics та продуктивність

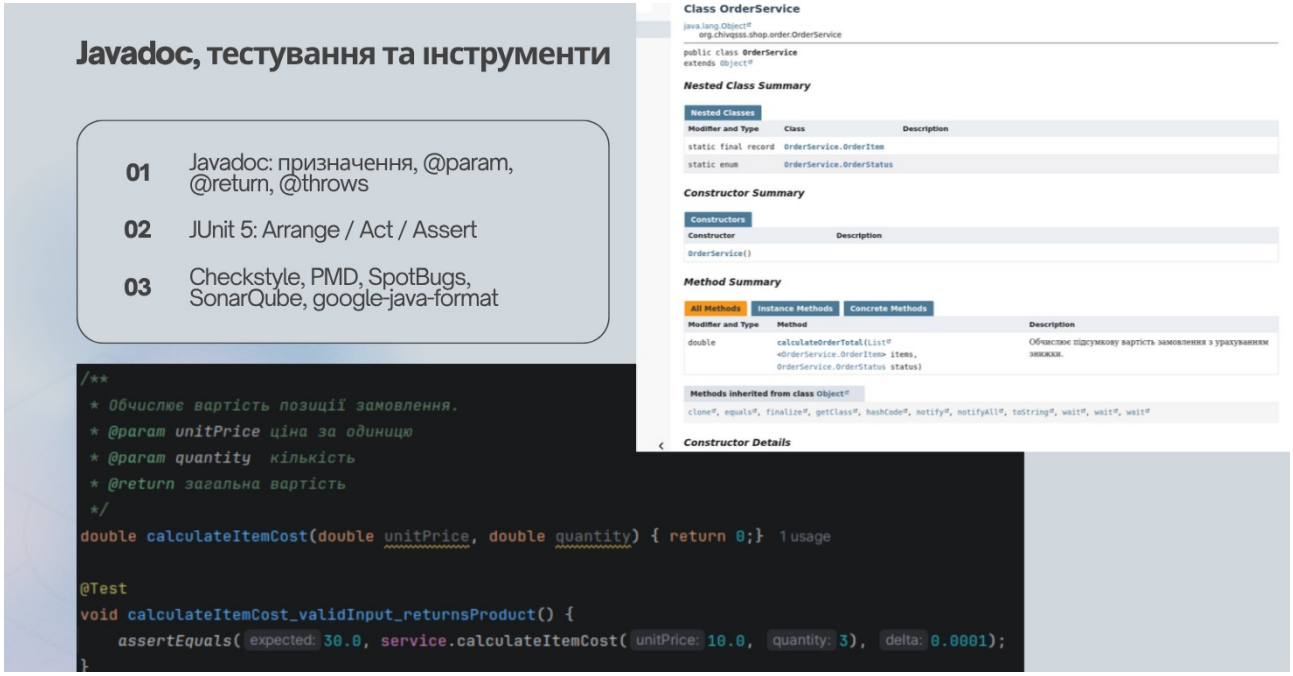


Рисунок Б.7 — Javadoc, тестування та інструменти аналізу

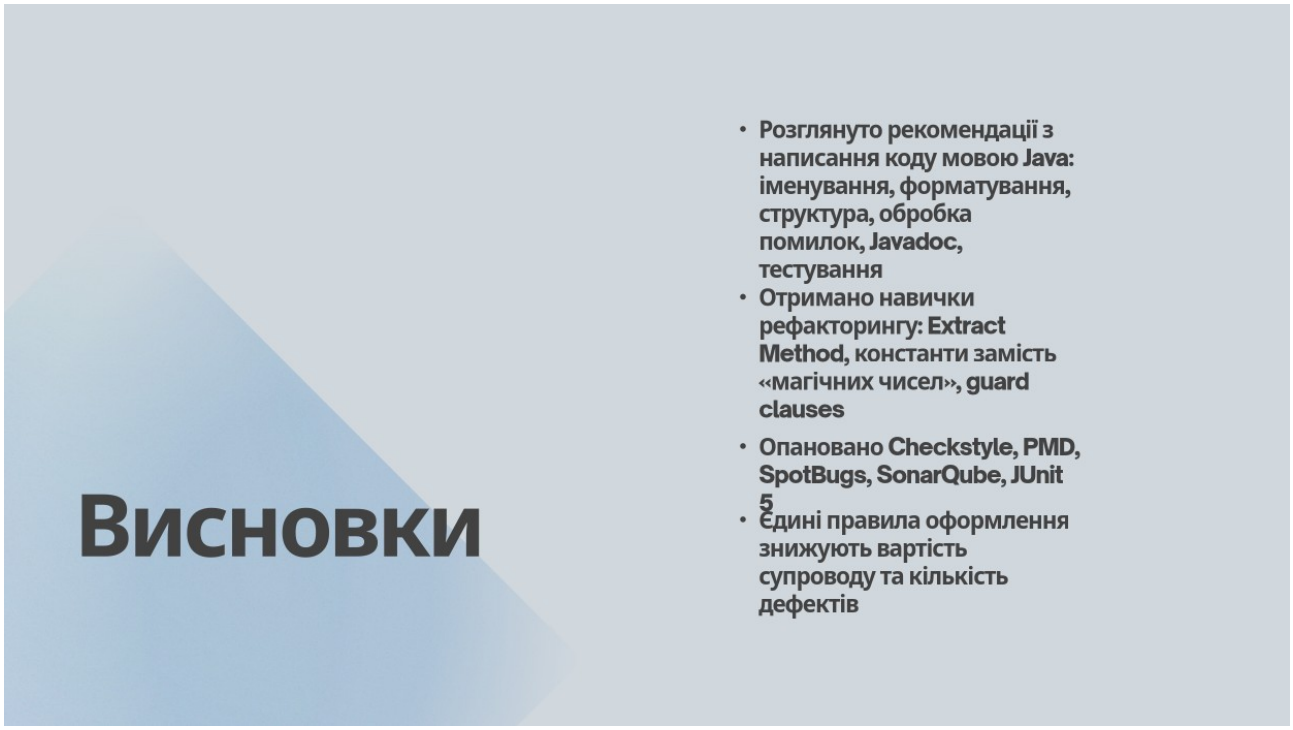


Рисунок Б.8 — Висновки

ДОДАТОК В

В.1 Приклад програмного коду до рефакторингу

GitHub репозиторій: https://github.com/NureKulinichOleksandr/ark-pzpi-24-6-kulinich-oleksandr/blob/main/Pract1/ark-pzpi-24-6-kulinich-oleksandr-pract1/order_service.java

```
 1  package org.chivqsss.shop.order;
 2
 3  import java.util.List;
 4  import java.util.Map;
 5
 6  public class order_service {
 7      public double process(List<Map<String, Object>> o, int s) {
 8          double t = 0;
 9          if (o != null) {
10              if (o.size() > 0) {
11                  for (int i = 0; i < o.size(); i++) {
12                      Map<String, Object> m = o.get(i);
13                      double p = (double) m.get("price");
14                      int q = (int) m.get("qty");
15                      t = t + p * q;
16                  }
17                  if (s == 2 && t > 1000) t = t * 0.9;
18              }
19          }
20          return t;
21      }
22  }
```

В.2 Приклад програмного коду після рефакторингу

GitHub репозиторій: https://github.com/NureKulinichOleksandr/ark-pzpi-24-6-kulinich-oleksandr/blob/main/Pract1/ark-pzpi-24-6-kulinich-oleksandr-pract1/OrderService.java

```
 1  package org.chivqsss.shop.order;
 2
 3  import java.util.List;
 4
 5  public class OrderService {
 6      private static final double DISCOUNT_THRESHOLD = 1000.0;
 7      private static final double DISCOUNT_RATE = 0.9;
 8
 9      /**
10       * Обчислює підсумкову вартість замовлення з урахуванням знижки.
11       *
12       * @param items  позиції замовлення, не може бути null
13       * @param status статус замовлення
14       * @return підсумкова вартість замовлення
15       */
16      public double calculateOrderTotal(List<OrderItem> items, OrderStatus status) {
17          if (items == null || items.isEmpty()) {
18              return 0.0;
19          }
20          double total = calculateItemsTotal(items);
21          return isDiscountApplicable(status, total) ? total * DISCOUNT_RATE : total;
22      }
```

```
23
24      private double calculateItemsTotal(List<OrderItem> items) {
25          double total = 0.0;
26          for (OrderItem item : items) {
27              total += item.price() * item.quantity();
28          }
29          return total;
30      }
31
32      private boolean isDiscountApplicable(OrderStatus status, double total) {
33          return status == OrderStatus.PAID && total > DISCOUNT_THRESHOLD;
34      }
35
36      public enum OrderStatus {NEW, PAID, SHIPPED, CANCELLED}
37
38      public record OrderItem(double price, int quantity) {
39      }
40  }
```

В.3 Модульні тести для перевірки збереження поведінки

GitHub репозиторій: https://github.com/NureKulinichOleksandr/ark-pzpi-24-6-kulinich-oleksandr/blob/main/Pract1/ark-pzpi-24-6-kulinich-oleksandr-pract1/OrderServiceTest.java

```
 1  import org.chivqsss.shop.order.OrderService;
 2  import org.junit.jupiter.api.Test;
 3
 4  import java.util.List;
 5
 6  import static org.junit.jupiter.api.Assertions.assertEquals;
 7
 8  class OrderServiceTest {
 9
10      private final OrderService service = new OrderService();
11
12      @Test
13      void calculateOrderTotal_emptyList_returnsZero() {
14          double result = service.calculateOrderTotal(
15                  List.of(), OrderService.OrderStatus.NEW);
16
17          assertEquals(0.0, result, 0.0001);
18      }
19
20      @Test
21      void calculateOrderTotal_paidAboveThreshold_appliesDiscount() {
22          List<OrderService.OrderItem> items =
23                  List.of(new OrderService.OrderItem(600.0, 2));
24
25          double result = service.calculateOrderTotal(
26                  items, OrderService.OrderStatus.PAID);
27
28          assertEquals(1080.0, result, 0.0001);
29      }
30
31      @Test
32      void calculateOrderTotal_notPaid_noDiscount() {
33          List<OrderService.OrderItem> items =
34                  List.of(new OrderService.OrderItem(600.0, 2));
35          double result = service.calculateOrderTotal(
36                  items, OrderService.OrderStatus.NEW);
37          assertEquals(1200.0, result, 0.0001);
```

```
38        }
39    }
```